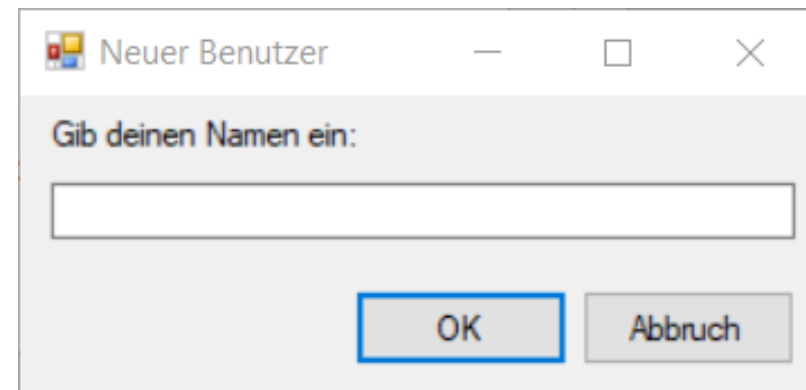


C# with GUI using the example of Windows Forms

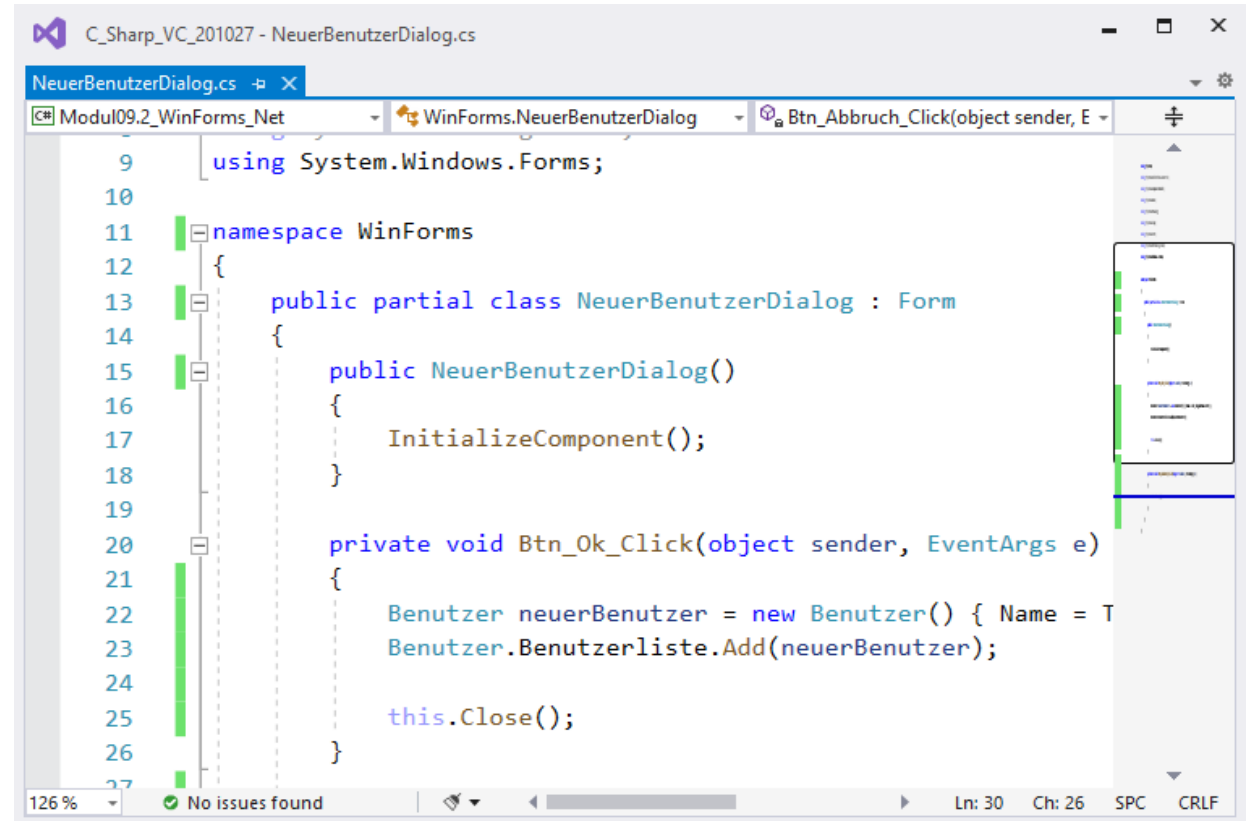
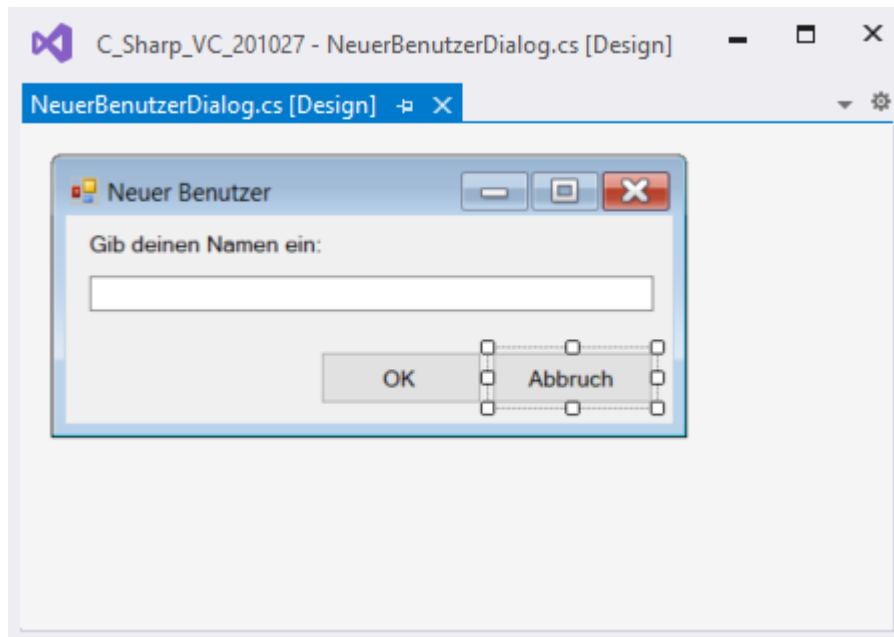
Windows Forms

- Windows Forms is the 'old' UI framework for desktop programs
 - 'new' UI framework: Windows Presentation Foundation (WPF)
- Form: Class for displaying a Windows window
- Event-based user interaction
- Design using visual designer



Windows Forms

- 'two-part' editing:
- GUI design in visual designer
- Logic in the 'code-behind'

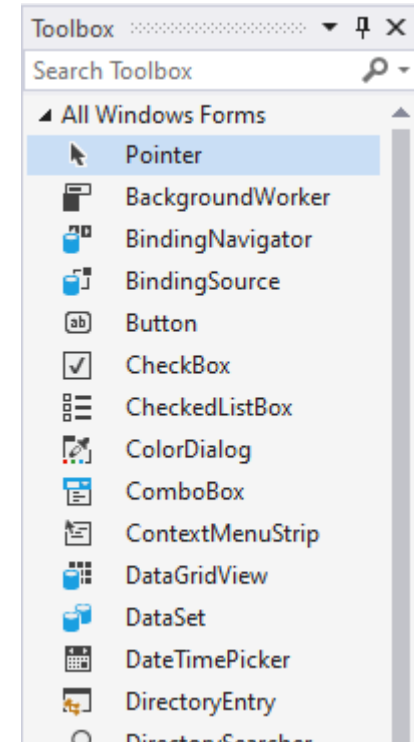
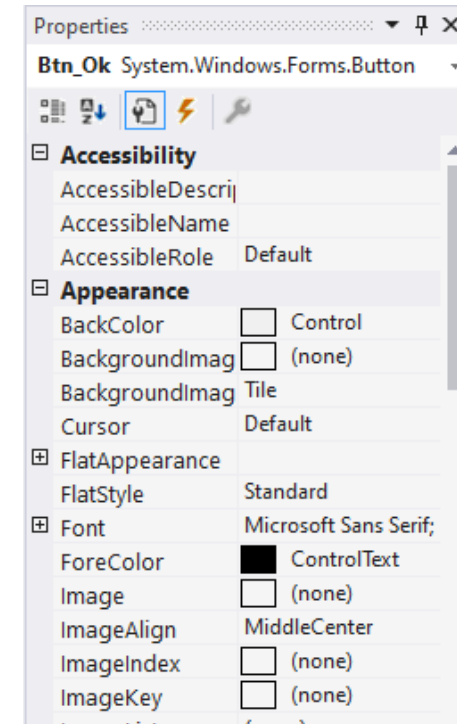


partial classes and methods

- classes and methods can also be split in different source files
- each source file contains parts of the type or method definition
- are combined when compiling the application
- each partial class must be marked as partial
- partial methods can only have the return value "void"
- keyword "partial" must be placed in front of the keyword "class" or "void"
- methods must be private and can be implemented only once

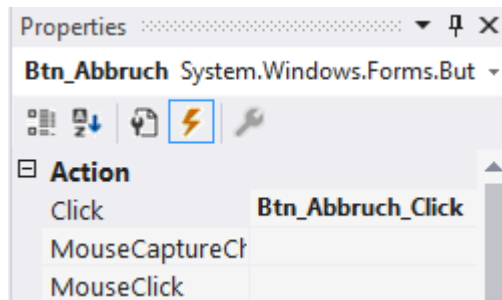
Windows Forms in Visual Studio

- Toolbox contains the GUI objects
- Drag'n'Drop for placement in the designer
- Properties window gives access to the properties of the selected GUI object



Properties and events

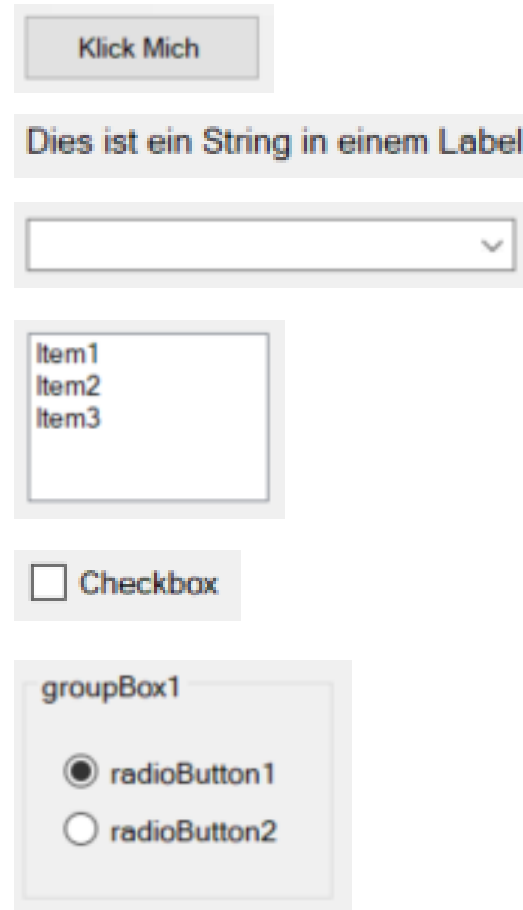
- In the 'Properties' window () there is a list of all accessible properties of the object selected in the Designer
- (Name) defines the variable identifier under which the object can be accessed
- In the 'Events' window () there is a list of available events
- Events must be linked to an EventHandler method in the code-behind



```
private void Btn_Abbruch_Click(object sender, EventArgs e)
{
    this.Close();
}
```

Important GUI objects

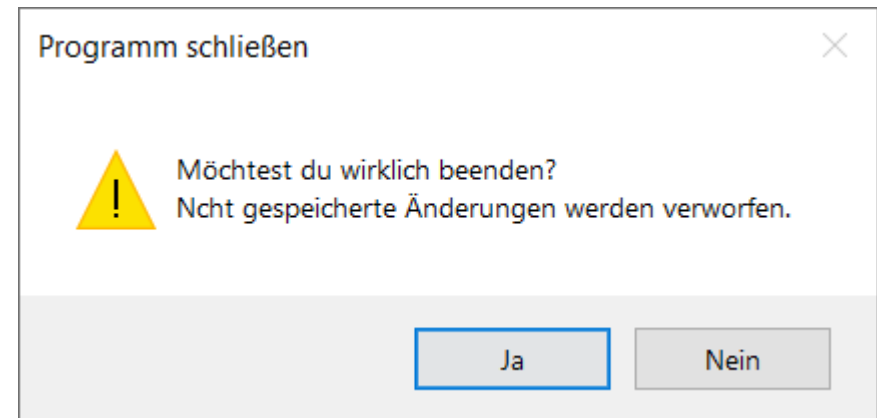
- Button
- Label
- ComboBox
- ListBox
- CheckBox
- RadioButton



- GUI classes inherit (indirectly) from `System.Windows.Forms.Control`
- In addition to the inheritors, each GUI class has specific, class-specific properties and events

MessageBox

- Message boxes are small, automatically generated dialog boxes
- For user queries or information
- Call is made via static method `MessageBox.Show()`
- Return of enum type `DialogResult` corresponding to clicked button



```
string dialogText = "Möchtest du wirklich beenden?\nNicht gespeicherte Änderungen werden verworfen.";
DialogResult result = MessageBox.Show(dialogText, "Programm schließen", MessageBoxButtons.YesNo, MessageBoxIcon.Warning);
if (result == DialogResult.Yes)
    this.Close();
```


Closing and opening forms

- `this.Close()` closes the current form
- `Form.Show()` opens a form as an equal window
- `Form.ShowDialog()` opens a form as a dialog window
- Dialog windows must be edited and closed before the user can edit another window
- `ShowDialog` returns `DialogResult`